Ex. No. 6	JavaScript Functions – Task Tracker
Date of Exercise	19.10.2026
GitHub Host Link	https://praneettigga.github.io/web-tech-lab/

Aim

To design and develop a Task Tracker web application with HTML, CSS and JavaScript, built from small, single-purpose JavaScript functions that add, complete, delete and filter tasks and redraw the page, in the same green and beige theme used in the earlier experiments.

Description

Functions: A function is a named, reusable block of code. Each job in the app has its own function (createTask, addTask, toggleTask, deleteTask, renderTasks), so every function does one thing, can be reused after any change, and a bug fixed in one place is fixed for every caller.

Parameters and return values: createTask(text) takes the task text as a parameter and uses return to hand a new task object { id, text, done } back to the caller.

State: The app keeps all its data in one place: a tasks array and the current filter. The updater functions change the state and renderTasks() redraws the screen from it.

Array methods and callbacks: Functions can be passed to other functions. filter keeps the items that pass a test (deleteTask, getVisible, countOpen), map transforms every item (toggleTask) and forEach runs a function for each item (renderTasks). Arrow functions such as t => !t.done are used as the callbacks.

DOM manipulation: document.getElementById and querySelectorAll select the placeholders in the page. document.createElement, appendChild and textContent build each list row (taskItem), and innerHTML clears the list before it is redrawn.

Event handling: addEventListener attaches handleSubmit to the form's submit event, and click/change listeners to the filter buttons, checkboxes and delete buttons. The function is passed to addEventListener, not called, so no brackets follow its name. event.preventDefault() stops the page from reloading on submit.

Local storage: saveAndRender() stores the tasks array in localStorage with JSON.stringify, and the page loads it back with JSON.parse, so the tasks remain after the page is refreshed.

External CSS: The page is styled by css/style.css using CSS variables, a card layout with a gold left border, flexbox for the form and task rows, and the .done and .active classes for finished tasks and the selected filter.

Procedure:

1. Select the application to be built: a Task Tracker with add, complete, delete and filter features.
2. Create a folder Expt6 with the sub-folders css and js.

3. Create index.html with the <form id="taskForm"> (text input and Add button), the filter buttons (All, Open, Finished), an empty <ul id="taskList"> and a <p id="counter">, and link css/style.css and js/app.js.

4. Create css/style.css, reusing the colour variables, font, card and button styles of the earlier experiments, and add styles for the filter buttons, task rows, finished tasks and the delete button.

5. In js/app.js, select the page elements and declare the state: the tasks array (loaded from localStorage) and the current filter.

6. Write createTask(text), which returns a new task object with an id, the text and done set to false.

7. Write addTask, toggleTask and deleteTask, each ending with a call to saveAndRender(), which saves the tasks to localStorage and calls renderTasks().

8. Write the helper functions getVisible() (applies the filter), countOpen() (counts tasks not done) and taskItem(t) (builds one list row with a checkbox, the text and a Delete button).

9. Write renderTasks(), which clears the list, appends a row for every visible task and updates the "left" counter.

10. Write handleSubmit(event) to read and trim the input, call addTask(text) when it is not empty and clear the field, and attach it with form.addEventListener("submit", handleSubmit).

11. Write setFilter(name) and attach it to the filter buttons, then call renderTasks() once to show the saved tasks on load.

12. Open index.html in a browser and test adding, completing, deleting and filtering tasks, and refresh the page to check that the tasks are kept.

13. Push the code to GitHub and host it using GitHub Pages to obtain the deployment link.

Program

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Task Tracker</title>
    <link rel="stylesheet" href="css/style.css" />
  </head>
  <body>
    <main class="page-shell">
      <section class="tracker-card">
        <h1>Task Tracker</h1>
        <p class="intro">Add a task, tick it off, or delete it.</p>

        <form id="taskForm">
          <input id="taskInput" type="text" placeholder="What needs doing?"
autocomplete="off" />
          <button type="submit">Add</button>
        </form>

        <div class="filters">
          <button type="button" class="filter active" data-filter="all">All</button>
          <button type="button" class="filter" data-filter="open">Open</button>
          <button type="button" class="filter" data-filter="done">Finished</button>
        </div>
```

```
        <ul id="taskList"></ul>
        <p id="counter" class="counter" aria-live="polite"></p>
      </section>
    </main>
    <script src="js/app.js"></script>
  </body>
</html>
```

css/style.css

```
:root {
  --ink: #314339;
  --heading: #244232;
  --accent: #c5a56a;
  --link: #496b55;
  --border: #d8ccb4;
  --surface: #fffdf8;
}

* { box-sizing: border-box; }

body {
  min-height: 100vh;
  margin: 0;
  padding: 30px 20px;
  color: var(--ink);
  font-family: Georgia, serif;
  background: #f4f0e7;
}

.page-shell { width: min(100%, 700px); margin: 0 auto; }

.tracker-card {
  padding: 28px;
  border: 1px solid var(--border);
  border-left: 5px solid var(--accent);
  border-radius: 10px;
  background: var(--surface);
  box-shadow: 0 4px 12px rgba(31, 77, 58, 0.08);
}

h1 { margin: 0; padding-bottom: 10px; border-bottom: 2px solid var(--border); color: var(--
heading); font-size: 2rem; }
.intro { margin: 16px 0 24px; line-height: 1.5; }

#taskForm { display: flex; gap: 10px; }
input {
  flex: 1;
  padding: 8px;
  border: 1px solid #b8a98a;
  border-radius: 4px;
  color: var(--ink);
  font: inherit;
  background: var(--surface);
}
input:focus { outline: 2px solid #e2c98e; border-color: var(--link); }

button {
  padding: 8px 18px;
  border: 1px solid var(--heading);
  border-radius: 4px;
  color: #fffaf0;
  background: var(--heading);
  cursor: pointer;
  font: 1rem Georgia, serif;
```

```
}
button:hover { background: var(--link); }

.filters { display: flex; gap: 8px; margin-top: 20px; }
.filter { padding: 6px 14px; color: var(--heading); background: #f8f5ef; border-color:
var(--border); font-size: 0.9rem; }
.filter:hover { background: #e7ede4; }
.filter.active { color: #fffaf0; background: var(--heading); border-color: var(--
heading); }

ul { margin: 18px 0 0; padding: 0; list-style: none; display: grid; gap: 10px; }
li { display: flex; align-items: center; gap: 12px; padding: 12px 14px; border: 1px solid
var(--border); border-radius: 4px; background: #f8f5ef; }
li span { flex: 1; line-height: 1.4; }
li.done span { color: #8a9a8f; text-decoration: line-through; }
li input[type="checkbox"] { flex: none; width: 18px; height: 18px; accent-color: var(--
heading); }
li .delete { margin: 0; padding: 4px 10px; color: var(--heading); background: transparent;
border-color: var(--border); font-size: 0.85rem; }
li .delete:hover { color: #fffaf0; background: #8a3b32; border-color: #8a3b32; }

.counter { margin: 18px 0 0; font-weight: bold; color: var(--heading); }
```

js/app.js

```
const form = document.getElementById("taskForm");
const input = document.getElementById("taskInput");
const list = document.getElementById("taskList");
const counter = document.getElementById("counter");
const filterButtons = document.querySelectorAll(".filter");

let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
let filter = "all";

function createTask(text) {
  return {
    id: Date.now(),
    text: text,
    done: false
  };
}

function addTask(text) {
  tasks.push(createTask(text));
  saveAndRender();
}

function toggleTask(id) {
  tasks = tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t));
  saveAndRender();
}

function deleteTask(id) {
  tasks = tasks.filter((t) => t.id !== id);
  saveAndRender();
}

function setFilter(name) {
  filter = name;
  filterButtons.forEach((button) => {
    button.classList.toggle("active", button.dataset.filter === name);
  });
  renderTasks();
}
```

```
function saveAndRender() {
  localStorage.setItem("tasks", JSON.stringify(tasks));
  renderTasks();
}

function getVisible() {
  if (filter === "open") return tasks.filter((t) => !t.done);
  if (filter === "done") return tasks.filter((t) => t.done);
  return tasks;
}

function countOpen() {
  return tasks.filter((t) => !t.done).length;
}

function taskItem(t) {
  const item = document.createElement("li");
  item.className = t.done ? "done" : "";

  const checkbox = document.createElement("input");
  checkbox.type = "checkbox";
  checkbox.checked = t.done;
  checkbox.addEventListener("change", () => toggleTask(t.id));

  const label = document.createElement("span");
  label.textContent = t.text;

  const remove = document.createElement("button");
  remove.type = "button";
  remove.className = "delete";
  remove.textContent = "Delete";
  remove.addEventListener("click", () => deleteTask(t.id));

  item.append(checkbox, label, remove);
  return item;
}

function renderTasks() {
  list.innerHTML = "";
  getVisible().forEach((t) => {
    list.appendChild(taskItem(t));
  });
  counter.textContent = countOpen() + " left";
}

function handleSubmit(event) {
  event.preventDefault();
  const text = input.value.trim();
  if (text !== "") {
    addTask(text);
  }
  input.value = "";
}

form.addEventListener("submit", handleSubmit);
filterButtons.forEach((button) => {
  button.addEventListener("click", () => setFilter(button.dataset.filter));
});

renderTasks();
```

Output

Task Tracker (empty)

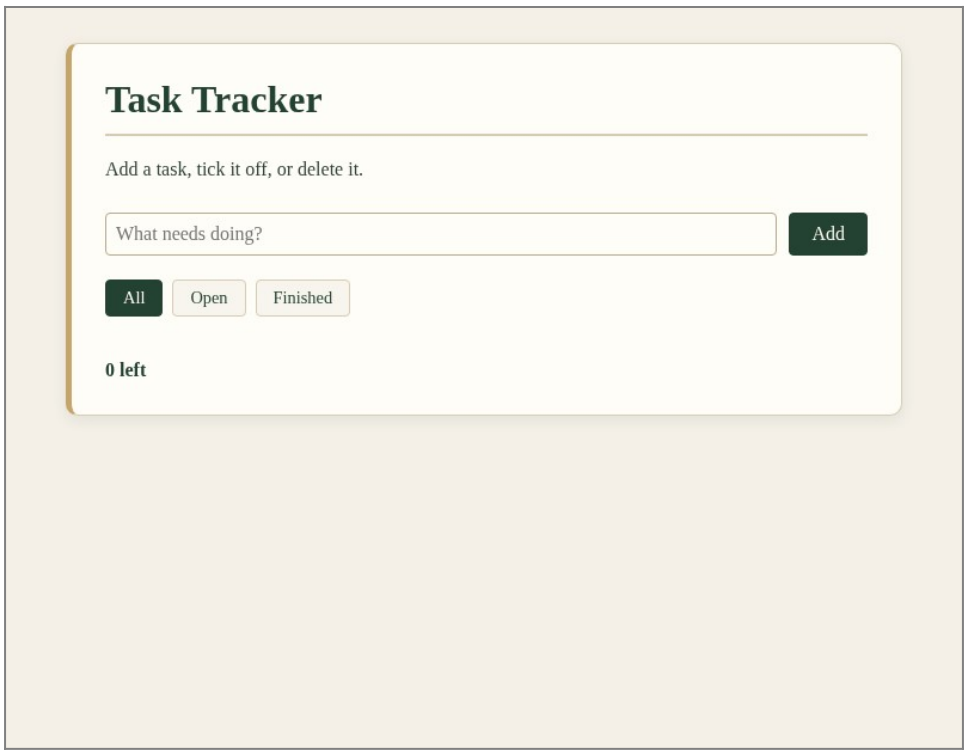


Figure 1: Task Tracker (empty state)

Task Tracker (with tasks)

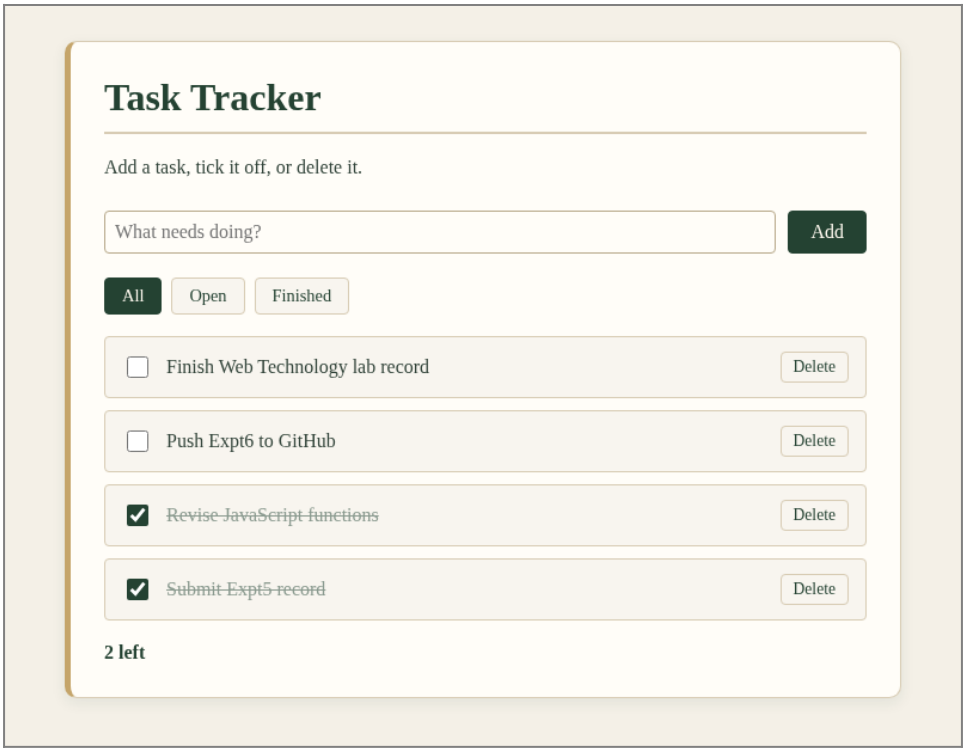


Figure 2: Task Tracker (with open and finished tasks)

Result

The Task Tracker was successfully designed and executed using JavaScript functions, array methods with callbacks, DOM manipulation, event handling and local storage. Tasks could be added, completed, deleted and filtered, the remaining count updated correctly, and the tasks were retained after reloading the page.